RENCANA PENGEMBANGAN ARSITEKTUR SISTEM AI-GEMPA BMKG

DAFTAR ISI

1. Ringkasan Eksekutif (Executive Summary)
2. Latar Belakang & Analisis Kondisi Sistem Saat Ini (Current State Analysis)
    - 2.1 Struktur Kode Eksisting
    - 2.2 Titik Lemah Arsitektur Eksisting (Pain Points)
    - 2.3 Urgensi dan Manfaat Pembaruan
3. Arsitektur Target & Prinsip Rekayasa (Target Architecture & Principles)
    - 3.1 Tujuh Prinsip Desain Utama
    - 3.2 Diagram Arsitektur Multi-Tier Terintegrasi
    - 3.3 Rencana Skema Bisnis Pay-Per-Use (Mata Uang IDR)
4. Rincian Komponen & Modul Baru
    - 4.1 Modul Katalog & Registri Penyedia Model (`catalog_module/`)
    - 4.2 Lapisan Domain Inti & Manajemen Profil (`core/` & `services/`)
    - 4.3 Mesin Pemantau Konsumsi Komputasi (`metering_module/`)
    - 4.4 Modul Penagihan & Faktur Otomatis (`billing_module/`)
    - 4.5 Abstraksi Algoritma AI Seismik (`providers/`)
    - 4.6 Antarmuka Pengguna Wizard Dinamis (`tews-ui-vue/`)
5. Spesifikasi Teknis Data & Antarmuka Layanan (API & Data Schema)
    - 5.1 Skema Koleksi Basis Data MongoDB
    - 5.2 Desain Aliran Pesan & Topik Apache Kafka
    - 5.3 Spesifikasi Kontrak REST API v2
6. Pohon Direktori & Struktur File Hasil Pengembangan
7. Tahapan Implementasi & Rencana Kerja (Implementation Roadmap)
    - 7.1 Matriks Fase, Durasi, dan Deliverables
    - 7.2 Strategi Migrasi Tanpa Gangguan Layanan (Zero Downtime Migration)
8. Analisis Risiko Teknis & Rencana Mitigasi

1. RINGKASAN EKSEKUTIF (EXECUTIVE SUMMARY)

Sistem peringatan dini dan pemrosesan gempa bumi terpadu **AI-GEMPA BMKG** telah berhasil membuktikan kapabilitas model pembelajaran mendalam (*deep learning*) dalam mendeteksi dan mengasosiasi fase gelombang seismik secara *real-time*. Namun demikian, seiring dengan kebutuhan perluasan operasional stasiun pemantau, integrasi multi-model dari berbagai vendor riset (misal: STA/LTA, PhaseNet, EQTransformer, GaMMA, dan PyOcto), serta tuntutan akuntabilitas utilisasi sumber daya komputasi, sistem eksisting memerlukan perombakan arsitektural (*refactoring* terstruktur).

Laporan ini menyajikan dokumen rancang bangun komprehensif untuk mentransformasi arsitektur perangkat lunak monolitik dan kaku saat ini menjadi sistem modern berbasis **Domain-Driven Design (DDD)** yang dilengkapi dengan dua pilar kemampuan baru:

1. **UI-Driven Pipeline Configuration:** Memungkinkan operator dan analis geofisika memilih, mengonfigurasi parameter, memvalidasi keterhubungan, serta mengaktifkan modul inferensi AI seismik secara dinamis langsung dari antarmuka web tanpa perlu menyentuh kode program atau me-restart kontainer server.
2. **Pay-Per-Use Metering & Billing System (Rupiah/IDR):** Mekanisme pelacakan konsumsi komputasi berbasis peristiwa (*event-driven metering*) yang mencatat setiap satuan beban inferensi gelombang seismik per *workspace*, menyajikan kalkulasi estimasi biaya langsung secara *real-time* saat perakitan pipa pemrosesan, serta menghasilkan faktur penagihan bulanan berbasis pemakaian riil dengan dukungan kuota gratis (*free tier*) dan kalkulasi PPN 11%.

Melalui rencana kerja bertahap, arsitektur ini menjamin portabilitas tinggi, pemisahan tanggung jawab kode yang bersih (*separation of concerns*), kompatibilitas penuh terhadap infrastruktur berjalan (*backward compatibility*), serta kesiapan sistem untuk diintegrasikan ke skema komputasi awan maupun portal publik BMKG.

2. LATAR BELAKANG & ANALISIS KONDISI SISTEM SAAT INI (CURRENT STATE ANALYSIS)

2.1 Struktur Kode Eksisting

Saat ini, repositori `ai-gempa-bmkg` terbagi dalam beberapa direktori parsial:

- `sispro-tews/`: Menampung modul pengolahan backend (`controller_module`, `seedlink_module`, `api_incoming_module`, `archiving_module`, dan `fdsn_module`).
- `AI-TEWS/`: Direktori terpisah yang memuat implementasi model AI (seperti `AI_modules_simple_ai` dan model eksperimental).
- `tews-ui-vue/`: Aplikasi frontend berbasis Vue 3 yang melayani pemantauan stasiun dan kejadian gempa.
- `development/`: Kumpulan dokumen desain dan prototipe skrip domain bisnis.

2.2 Titik Lemah Arsitektur Eksisting (Pain Points)

Berdasarkan audit teknis mendalam terhadap kode sumber dan alur operasional, ditemukan sejumlah kendala mendasar:

No	Gejala Masalah (Issue)	Dampak terhadap Sistem & Operasional	Akar Penyebab Teknis (Root Cause)
1	**Duplikasi Entry Point Server**	Kebingungan tim pengembang, disparitas fitur antar lingkungan, dan risiko tinggi salah eksekusi file produksi.	Terdapat banyak file di `controller_module` seperti `app.py`, `app1.py`, `controller_api_app.py`, dan `controller_web_socket_redis.py` tanpa satu gerbang masuk resmi.
2	**Struktur Service Datar (*Flat Structure*)**	Sulit melakukan ekspansi modul baru; dependensi antar lapisan kode menjadi kusut (*spaghetti dependency*).	Seluruh *business logic* ditumpuk pada satu folder `services/` di dalam `controller_module` tanpa pemisahan batas domain (*bounded context*).
3	**Model AI dan Parameter Bersifat *Hardcoded***	Pergantian model (misal dari STA/LTA klasik ke PhaseNet GPU) mewajibkan modifikasi file internal, pengeditan variabel lingkungan, dan restart kontainer.	Ketiadaan lapisan abstraksi konfigurasi (*configuration abstraction layer*) dan tidak adanya penyedia antarmuka (*provider interface*).
4	**Ketiadaan Sistem Pelacakan Penggunaan (*Metering*)**	Manajemen BMKG tidak memiliki visibilitas data mengenai volume inferensi harian yang dikonsumsi oleh tiap stasiun atau unit kerja regional.	Setiap modul data plane memproses data seismik tanpa memancarkan metrik atau peristiwa penggunaan (*usage event*).

No	Gejala Masalah (Issue)	Dampak terhadap Sistem & Operasional	Akar Penyebab Teknis (Root Cause)
5	**Ketiadaan Sistem Penetapan Biaya (*Pricing/Billing*)**	Sistem tidak dapat dioperasikan dalam skema multi-tenancy, komputasi bersama lintas instansi, atau layanan data berbayar (*Pay-Per-Use*).	Arsitektur lama dibangun murni sebagai pengolah sinyal statis, tanpa domain finansial/biaya komputasi.
6	**Ketidakmampuan Pelacakan Versi Konfigurasi (*Config Versioning*)**	Sulit melacak perubahan parameter seismologi, tidak ada mekanisme *rollback* konfigurasi yang aman, dan risiko kegagalan pipa deteksi akibat parameter cacat.	Dokumen profil pipa pemrosesan (*pipeline profile*) tidak memiliki nomor revisi (*revision hash*) dan status tata kelola (*draft/active/archived*).
7	**Kopling Erat Antara Antarmuka dan Backend**	Frontend Vue hanya menampilkan visualisasi baku dan tidak berdaya mengubah parameter alur analisis gempa secara mandiri.	Seluruh logika alur dikendalikan secara mutlak dan kaku oleh konfigurasi statis di sisi server backend.

2.3 Urgensi dan Manfaat Pembaruan

Pembaruan arsitektur ini bukan sekadar perapihan kode (*cosmetic refactoring*), melainkan fondasi strategis untuk:

- **Ketahanan Operasional:** Menghilangkan *downtime* saat kalibrasi parameter seismik melalui mekanisme *hot-reload parameterization*.
- **Efisiensi Anggaran Infrastruktur:** Menyajikan metrik konsumsi GPU/CPU per detik pemrosesan sinyal sehingga pengalokasian perangkat keras server BMKG terukur berbasis data riil.

- **Ekosistem Terbuka:** Memberi kemudahan bagi tim peneliti eksternal untuk memasang model AI deteksi gempa baru secara *plug-and-play* cukup dengan memenuhi kontrak antarmuka *provider*.

3. ARSITEKTUR TARGET & PRINSIP REKAYASA (TARGET ARCHITECTURE & PRINCIPLES)

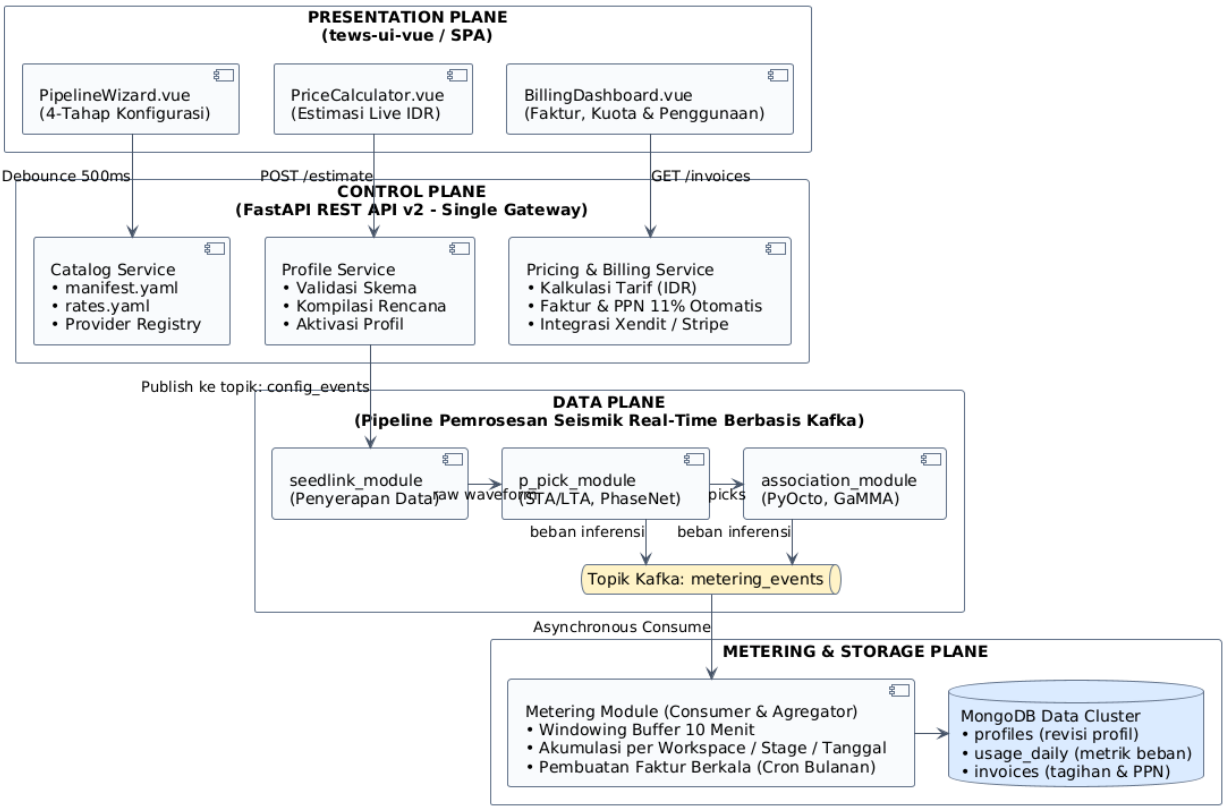
3.1 Tujuh Prinsip Desain Utama

Untuk mengatasi seluruh *pain points* di atas, rekayasa ulang mengadopsi 7 pilar standar industri:

1. **Domain-Driven Design (DDD):** Membagi kode berdasarkan kapabilitas bisnis (*Core*, *Services*, *Infrastructure*, *API*, dan *Providers*), bukan sekadar pemisahan lapisan teknis.
2. **Provider Abstraction Pattern:** Setiap algoritma seismologi (deteksi fase gelombang, pengelompokan asosiasi, kalkulasi magnitudo) dibungkus dalam antarmuka terstandarisasi (`BaseProvider`), menjadikannya modular dan dapat diganti kapan saja.
3. **Event-Driven Data & Metering Plane:** Setiap mutasi konfigurasi dan aktivitas inferensi memancarkan pesan ke Apache Kafka (`config_events` dan `metering_events`) secara asinkron tanpa membebani latensi pemrosesan sinyal utama.
4. **Configuration as Code & Immutability:** Profil pipa pemrosesan didefinisikan sebagai dokumen terstruktur berversi (*revision hash* SHA-256 yang deterministik) berstatus *draft*, *validated*, *active*, atau *archived*.
5. **API-First & Strict Contract:** Seluruh komunikasi data antar modul divalidasi ketat menggunakan skema Pydantic dan dokumentasi OpenAPI/Swagger v2 otomatis.
6. **Isolated Testability:** Setiap unit kalkulasi tarif, agregasi penggunaan, dan kompilasi profil dapat diuji secara mandiri dengan *unit test* tanpa ketergantungan koneksi basis data aktif.
7. **Comprehensive Observability & Audit Trail:** Seluruh aksi konfigurasi, estimasi biaya, dan volume inferensi tersimpan rapi untuk kebutuhan audit operasional BMKG.

3.2 Diagram Arsitektur Multi-Tier Terintegrasi

Sistem baru membagi tanggung jawab ke dalam 4 bidang (*planes*) yang terkoordinasi:



3.3 Rencana Skema Bisnis Pay-Per-Use (Mata Uang IDR)

[Pada pembahasan ini masih berupa rencana awal yang nantinya dapat disesuaikan dengan kebutuhan serta strategi keputusan bisnis yang akan diambil selanjutnya]

Menjawab kebutuhan tata kelola sumber daya komputasi modern, sistem tidak menerapkan biaya langganan kaku (*subscription flat-rate*), melainkan murni berbasis volume inferensi riil:

1. **Satuan Pemakaian Terukur (*Metered Units*):**
    - **P-Pick Stage:** Dihitung per 1.000 picks fase gelombang yang terdeteksi.
    - **Association Stage:** Dihitung per 1.000 klaster asosiasi kejadian gempa.
    - **Location & Magnitude:** Dihitung per estimasi solusi hiposenter terkonfirmasi.
2. **Skema Kuota Gratis Bulanan (*Monthly Free Tier*):** Setiap *workspace* berhak atas kuota bebas biaya setiap bulan untuk mendukung riset dasar BMKG (contoh: 10.000 picks gratis untuk STA/LTA, 5.000 picks untuk PhaseNet). Pembebanan biaya hanya berlaku saat pemakaian melampaui ambang batas kuota.
3. **Kepatuhan Pajak Nasional:** Sistem secara otomatis menambahkan PPN 11% pada faktur resmi, lengkap dengan rincian subtotal per modul.
4. **Tarif Fleksibel Tanpa Kompilasi (*Dynamic Rates via YAML*):** Manajemen dapat memperbarui tarif satuan IDR pada file `rates.yaml` kapan saja tanpa perlu mengubah baris kode atau melakukan *downtime* server.

Berikut adalah tabel matriks tiering model dan tarif acuan sistem:

Tahapan Pemrosesan	Provider / Algoritma	Tier Komputasi	Kuota Gratis Bulanan	Tarif Satuan (IDR)	Karakteristik Performa
P-Wave Picking	`sta_lta`	CPU Baseline	10.000 picks	Rp 50 / 1K picks	Eksekusi deterministik cepat, latensi rendah
P-Wave Picking	`phasenet`	GPU Standard	5.000 picks	Rp 500 / 1K picks	Deep 1D U-Net, presisi tinggi pada noise
P-Wave Picking	`eqtransformer`	GPU Premium	1.000 picks	Rp 1.000 / 1K picks	Deteksi simultan P, S, dan estimasi waktu tiba
Association	`dbscan_simple`	CPU Basic	5.000 klaster	Rp 100 / 1K klaster	Pengelompokan spasial cepat
Association	`gamma`	AI Advanced	500 klaster	Rp 500 / 1K klaster	Gaussian Mixture Model probabilistik tinggi
Association	`pyocto`	High Throughput	500 klaster	Rp 750 / 1K klaster	Optimal untuk jaringan sensor densitas rapat
Location & Mag	`locmag_geiger`	CPU Fast	2.000 kejadian	Rp 150 / 1K solusi	Inversi linier waktu rambat klasik
Location & Mag	`nonlinloc`	Non-Linear Prob	500 kejadian	Rp 800 / 1K solusi	Distribusi densitas probabilitas global

4. RINCIAN KOMPONEN & MODUL BARU

Untuk mewujudkan arsitektur target di atas, dilakukan pembentukan modul-modul modular baru dengan pemisahan tugas yang tegas:

4.1 Modul Katalog & Registri Penyedia Model (`catalog_module/`)

Modul ini bertindak sebagai sumber kebenaran tunggal (*single source of truth*) yang mempublikasikan daftar algoritma yang tersedia, spesifikasi perangkat keras minimum, metrik akurasi historis, serta tarif harga:

- `provider_catalog.py`: Komponen pemuat dokumen `manifest.yaml` dan `rates.yaml` ke dalam memori aplikasi saat inisialisasi awal.
- `manifest.yaml`: Berkas deklaratif yang mendaftarkan seluruh penyedia model per tahapan.
- `rates.yaml`: Tabel harga resmi dalam denominasi Rupiah.

4.2 Lapisan Domain Inti & Manajemen Profil (`core/` & `services/`)

Memisahkan logika bisnis dari protokol jaringan dan basis data:

- `core/profile.py`: Mendefinisikan struktur data dokumen pipa pemrosesan (`Pipeline`, `StageConfig`, `SinkConfig`).
- `core/pricing.py`: Memuat struktur entitas kalkulasi tarif (`PriceEstimate`, `InvoiceBreakdown`).
- `services/profile_service.py`: Bertanggung jawab atas siklus hidup profil:
    - *Drafting:* Menyimpan rancangan sementara yang sedang disusun di antarmuka web.
    - *Validation:* Memverifikasi bahwa setiap model yang dipilih memiliki sumber data yang valid dan kompatibel.
    - *Compilation:* Mengompilasi profil menjadi rencana eksekusi sistem (menghasilkan *revision hash* SHA-256 yang unik dan tidak dapat diubah).
    - *Activation:* Menerbitkan *event* aktivasi ke Kafka untuk diterapkan oleh seluruh *worker*.
- `services/pricing_service.py`: Mesin hitung yang mengeksekusi rumus penagihan, memotong kuota gratis bulanan, menambahkan PPN 11%, dan menyusun struktur tagihan.

4.3 Mesin Pemantau Konsumsi Komputasi (`metering_module/`)

Merupakan layanan berlatar belakang (*background daemon*) independen:

- `consumer.py`: Mendengarkan aliran data dari Kafka pada topik `metering_events`. Setiap kali modul inferensi (misal `p_pick_module`) menyelesaikan 1 batch gelombang, modul tersebut mengirimkan tiket penggunaan (memuat: `workspace_id`, `stage`, `provider_id`, `unit_count`, `duration_ms`).
- `aggregator.py`: Menerapkan jendela waktu (*windowing buffer*) selama 10 menit untuk mengakumulasi tiket penggunaan sebelum menuliskan rangkumannya secara efisien ke dalam koleksi MongoDB `usage_daily`.

4.4 Modul Penagihan & Faktur Otomatis (`billing_module/`)

Menangani tata kelola keuangan sistem:

- `invoice_generator.py`: Skrip *cron job* terjadwal yang dieksekusi setiap akhir siklus penagihan (akhir bulan) untuk menarik data dari `usage_daily` dan mengunci total pemakaian ke dalam koleksi `invoices`.
- `payment_handler.py`: Titik akhir *webhook* untuk menerima notifikasi pelunasan dari penyedia gerbang pembayaran digital Indonesia (seperti Xendit atau Stripe).

4.5 Abstraksi Algoritma AI Seismik (`providers/`)

Mengubah modul AI dari konfigurasi kaku menjadi pustaka *pluggable*:

- `base.py`: Antarmuka abstrak `BaseProvider` yang mewajibkan implementasi fungsi `initialize()`, `infer()`, dan `health_check()`.
- Direktori sub-tahap: `p_pick/`, `association/`, dan `locmag/` memuat adaptor pembungkus masing-masing algoritma sehingga runtime utama tidak peduli apakah inferensi dijalankan di PyTorch, TensorFlow, maupun C++.

4.6 Antarmuka Pengguna Wizard Dinamis (`tews-ui-vue/`)

Peningkatan antarmuka pengguna pada dashboard Vue 3:

- **`PipelineWizard.vue` (4-Tahap Alur Kerja):**
  1. *Pilih Mode:* Mode Default (konfigurasi stabil BMKG) atau Mode Kustom.
  2. *Konfigurasi Tahapan:* Mengaktifkan/menonaktifkan filter gelombang, P-Pick, S-Pick, asosiasi, atau kalkulasi lokasi.
  3. *Pemilihan Model:* Memilih model AI dari galeri katalog lengkap dengan visualisasi metrik akurasi (*precision, recall, latency*).
  4. *Tinjau & Aktifkan:* Evaluasi keterhubungan antar model dan konfirmasi penerapan.
- **`PriceCalculator.vue` (Sidebar Estimasi Real-Time):**
  - Panel interaktif di sisi kanan wizard yang bereaksi secara instan saat operator mengklik model AI yang berbeda.
  - Mengirimkan *request* tertunda (*debounced* 500ms) ke endpoint `/api/v2/pricing/estimate` sehingga estimasi biaya bulanan dalam Rupiah langsung tampil sebelum profil disimpan.
- **`BillingDashboard.vue`:**
  - Menyajikan grafik pemakaian harian riil, sisa kuota gratis bulanan, riwayat faktur, dan opsi unduh bukti tagihan PDF.

5. SPESIFIKASI TEKNIS DATA & ANTARMUKA LAYANAN (API & DATA SCHEMA)

5.1 Skema Koleksi Basis Data MongoDB

Struktur koleksi baru dirancang untuk skalabilitas data tinggi dan efisiensi pengindeksan:

A. Koleksi `profiles`

Menyimpan seluruh spesifikasi pipa pengolahan:

```
{
  "_id": { "$oid": "670f1a2b8e1a2b3c4d5e6f01" },
  "workspace_id": "bmkg-pusat",
  "name": "Pipeline Deteksi Cepat Wilayah Megathrust Jawa",
  "mode": "custom",
  "stages": {
    "p_pick": {
      "provider_id": "phasenet",
      "enabled": true,
      "params": { "threshold": 0.85, "sampling_rate": 20 }
    },
    "association": {
      "provider_id": "pyocto",
      "enabled": true,
      "params": { "min_stations": 4, "time_tolerance_sec": 3.0 }
    },
    "locmag": {
      "provider_id": "nonlinloc",
      "enabled": true,
      "params": { "velocity_model": "iasp91" }
    }
  },
  "rev": "sha256:7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284addd200126d
9069",
  "status": "active",
  "created_at": "2026-10-15T08:30:00Z",
  "activated_at": "2026-10-15T09:00:00Z",
  "created_by": "operator_geofisika"
}
```

B. Koleksi `usage_daily`

Menyimpan akumulasi beban inferensi harian per modul (diindeks secara komposit pada `workspace_id` + `date`):

```
{
  "_id": { "$oid": "670f9b3c8e1a2b3c4d5e6f02" },
  "workspace_id": "bmkg-pusat",
  "date": "2026-10-18",
```

```
  "stage": "p_pick",
  "provider_id": "phasenet",
  "profile_rev": "sha256:7f83b1657ff1fc53b92dc18148a1d65dfc2d4b1fa3d677284add
d200126d9069",
  "total_units": 45000,
  "event_count": 450,
  "total_duration_ms": 3825000,
  "created_at": "2026-10-18T23:59:59Z"
}
```

C. Koleksi `invoices`

Menyimpan rincian tagihan resmi bulanan:

```
{
  "_id": { "$oid": "6710a12e8e1a2b3c4d5e6f03" },
  "workspace_id": "bmkg-pusat",
  "invoice_number": "INV/BMKG/202610/0042",
  "period_start": "2026-10-01",
  "period_end": "2026-10-31",
  "breakdown": {
    "p_pick": {
      "provider_id": "phasenet",
      "units_total": 45000,
      "units_free": 5000,
      "units_charged": 40000,
      "rate_per_unit_idr": 500,
      "subtotal_idr": 20000000
    },
    "association": {
      "provider_id": "pyocto",
      "units_total": 3500,
      "units_free": 500,
      "units_charged": 3000,
      "rate_per_unit_idr": 750,
      "subtotal_idr": 2250000
    }
  },
  "subtotal_idr": 22250000,
  "ppn_11_percent_idr": 2447500,
  "total_idr": 24697500,
  "status": "paid",
  "payment_method": "va_bni",
  "paid_at": "2026-11-02T10:15:00Z",
  "created_at": "2026-11-01T00:00:00Z"
}
```

5.2 Desain Aliran Pesan & Topik Apache Kafka

Sistem menggunakan 2 topik Kafka baru untuk memastikan pemrosesan tidak membebani jalur seismik:

1. **Topik `config_events`:** Dipancarkan oleh `ProfileService` saat sebuah profil diaktifkan.
    - *Payload:* ID profil, revisi hash, dan daftar konfigurasi parameter per tahap.
    - *Consumer:* Seluruh kontainer inferensi AI mendengarkan topik ini untuk memperbarui bobot atau parameter deteksi secara *hot-reload*.
2. **Topik `metering_events`:** Dipancarkan oleh setiap *worker* inferensi setelah mengeksekusi data gelombang.
    - *Payload:* ID workspace, nama tahap, ID provider, jumlah unit inferensi, dan waktu eksekusi.
    - *Consumer:* `metering_module` untuk proses agregasi periodik.

5.3 Spesifikasi Kontrak REST API v2

Seluruh interaksi baru dikonsolidasikan pada gerbang API v2:

Endpoint Path	Metode HTTP	Deskripsi Fungsional	Parameter / Payload Utama
`/api/v2/models`	`GET`	Mengambil katalog seluruh model AI, status kesiapan, metrik, dan tabel harga.	Query: `?stage=p_pick` (opsional)
`/api/v2/profiles`	`GET`	Menampilkan seluruh riwayat konfigurasi pipa pemrosesan per workspace.	Header: `X-Workspace-ID`
`/api/v2/profiles`	`POST`	Menyimpan draf konfigurasi profil baru sebelum diaktifkan.	Body JSON: `name, mode, stages, sinks`
`/api/v2/profiles/{id}`	`GET`	Mengambil rincian detail konfigurasi dan status validasi profil tertentu.	Path: `id` (ObjectId)

Endpoint Path	Metode HTTP	Deskripsi Fungsional	Parameter / Payload Utama
`/api/v2/profiles/{id}/validate`	`POST`	Melakukan audit teknis terhadap kelayakan keterhubungan konfigurasi.	Path: `id`
`/api/v2/profiles/{id}/activate`	`POST`	Mengompilasi hash deterministik dan mempublikasikan aktivasi ke Kafka.	Path: `id`
`/api/v2/pricing/estimate`	`POST`	Menghitung simulasi biaya IDR secara instan berdasarkan draf di UI.	Body JSON: `stages`, `estimated_volume`
`/api/v2/pricing/rates`	`GET`	Memperoleh daftar tarif satuan resmi yang sedang aktif.	-
`/api/v2/pricing/usage/{ws}`	`GET`	Menampilkan metrik pemakaian berjalan pada bulan kalender aktif.	Path: `ws` (Workspace ID)
`/api/v2/billing/invoices`	`GET`	Menampilkan daftar faktur tagihan bulanan resmi.	Query: `?status=paid`
`/api/v2/billing/invoices/{id}/pdf`	`GET`	Mengunduh lembar faktur resmi berformat PDF.	Path: `id`

6. POHON DIREKTORI & STRUKTUR FILE HASIL PENGEMBANGAN

Berikut adalah gambaran menyeluruh pohon direktori proyek setelah rancangan ini diimplementasikan secara tuntas. Setiap komponen diberi keterangan status:

- [BARU] : Modul/file yang dibangun dari awal.
- [DITINGKATKAN] : File eksisting yang dimodifikasi strukturnya.
- [DIRELOKASI] : File lama yang dirapikan ke dalam paket domain barunya.
- [EKSISTING] : File inti lama yang tetap dipertahankan.

```
ai-gempa-bmkg/
│
├── sispro-tews/                          # [DITINGKATKAN] Paket Backend Pemrosesan Seismik Utama
│   ├── api/                              # [BARU] Gerbang Tunggal REST API v2 (FastAPI)
│   │   ├── __init__.py
│   │   ├── main.py                       # [BARU] Gerbang aplikasi utama resmi (menggantikan app.py lama)
│   │   ├── dependencies.py               # [BARU] Dependency injection (Auth, DB, Kafka, Workspace context)
│   │   ├── routes/                       # [BARU] Pengelompokan Endpoint Berbasis Domain
│   │   │   ├── __init__.py
│   │   │   ├── models_routes.py          # [BARU] GET /api/v2/models (Katalog Model & Metrik)
│   │   │   ├── profiles_routes.py        # [BARU] CRUD /api/v2/profiles & /activate
│   │   │   ├── pricing_routes.py         # [BARU] POST /api/v2/pricing/estimate & /usage
│   │   │   ├── billing_routes.py         # [BARU] GET /api/v2/billing/invoices & Webhook
│   │   │   ├── stations_routes.py        # [DIRELOKASI] Endpoint monitoring stasiun BMKG
│   │   │   ├── events_routes.py          # [DIRELOKASI] Endpoint data katalog gempa
│   │   │   └── users_routes.py           # [DIRELOKASI] Endpoint autentikasi pengguna
│   │   └── schemas/                      # [BARU] Validasi Kontrak Data Pydantic
│   │       ├── __init__.py
│   │       ├── profile_schema.py         # Skema validasi pembuatan & aktivasi profil
│   │       ├── pricing_schema.py         # Skema estimasi tarif dan respons kalkulator
│   │       ├── billing_schema.py         # Skema dokumen faktur dan rincian pemakaian
│   │       └── catalog_schema.py         # Skema penyedia model dan metrik
│   │
│   ├── core/                             # [BARU] Entitas Domain Murni (Agnostik dari Database/Framework)
│   │   ├── __init__.py
│   │   ├── profile.py                    # Entitas Pipeline, StageConfig, SinkConfig, Revision
│   │   ├── pricing.py                    # Entitas PriceEstimate, InvoiceBreakdown, UnitRate
│   │   ├── artifacts.py                  # Entitas artefak seismik (Waveform, Pick, Cluster, Event)
│   │   └── events.py                     # Definisi Domain Event (UsageEvent, ConfigActivatedEvent)
│   │
│   ├── services/                         # [BARU] Lapisan Logika Bisnis & Orkestrasi Sistem
│   │   ├── __init__.py
│   │   ├── profile_service.py            # Siklus hidup profil (Draft, Validasi, Kompilasi, Aktivasi)
│   │   ├── pricing_service.py            # Formula kalkulasi tarif IDR, pemotongan kuota, dan PPN 11%
│   │   ├── metering_service.py           # Pemrosesan dan agregasi data konsumsi inferensi
│   │   ├── catalog_service.py            # Pemuat katalog model seismik dari manifest
│   │   ├── station_service.py            # [DIRELOKASI dari controller_module lama]
│   │   └── event_service.py              # [DIRELOKASI dari controller_module lama]
│   │
│   ├── infrastructure/                   # [BARU] Lapisan Adaptor Infrastruktur & Database
│   │   ├── __init__.py
│   │   ├── repositories/                 # Akses Operasi Basis Data MongoDB
│   │   │   ├── __init__.py
│   │   │   ├── profile_repository.py     # Operasi koleksi 'profiles'
│   │   │   ├── usage_repository.py       # Operasi agregasi koleksi 'usage_daily'
│   │   │   ├── invoice_repository.py     # Operasi koleksi 'invoices'
│   │   │   └── station_repository.py     # Operasi data stasiun seismik
│   │   ├── kafka/                        # Adaptor Komunikasi Pesan Kafka
│   │   │   ├── __init__.py
│   │   │   ├── producers.py              # Pemancar pesan (config_events, waveform_stream)
│   │   │   ├── consumers.py              # Konsumen pesan latar belakang
│   │   │   └── topics.py                 # Definisi nama dan partisi topik Kafka
│   │   ├── cache/                        # Adaptor Redis Cache & State
│   │   │   ├── __init__.py
│   │   │   └── redis_client.py           # State sinkronisasi WebSocket & cache kuota
│   │   └── storage/                      # Klien Basis Data Fisik
│   │       ├── __init__.py
│   │       └── mongodb_client.py         # Koneksi cluster MongoDB
│   │
│   ├── catalog_module/                   # [BARU] Registri Mandiri Penyedia Model & Tarif
│   │   ├── __init__.py
│   │   ├── provider_catalog.py           # Loader in-memory registry
│   │   └── data/
│   │       ├── manifest.yaml             # Deklarasi model, tahapan, dan spesifikasi hardware
│   │       └── rates.yaml                # Dokumen tarif resmi (IDR) per model tier
│   │
│   ├── metering_module/                  # [BARU] Daemon Akumulator Penggunaan Komputasi
│   │   ├── __init__.py
│   │   ├── consumer.py                   # Konsumen Kafka untuk topik 'metering_events'
│   │   ├── aggregator.py                 # Windowing 10 menit & agregasi harian per workspace
```

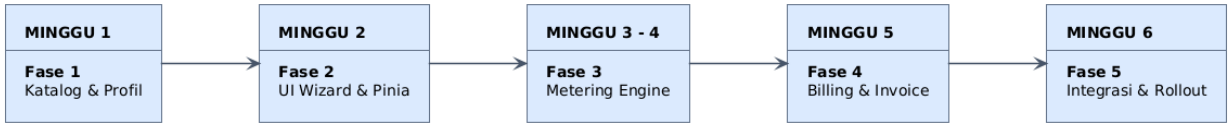
```
│   │   └── config.py                       # Konfigurasi buffer dan limit memori
│   │
│   ├── billing_module/                     # [BARU] Daemon Penagihan & Integrasi Gateway Pembayaran
│   │   ├── __init__.py
│   │   ├── invoice_generator.py            # Cron job pembuat faktur otomatis akhir bulan
│   │   ├── payment_handler.py              # Pengelola callback webhook Xendit/Stripe
│   │   └── config.py                       # Kunci otorisasi gateway pembayaran
│   │
│   ├── providers/                          # [BARU] Abstraksi Pustaka Algoritma & Inferensi AI
│   │   ├── __init__.py
│   │   ├── base.py                         # Antarmuka abstrak BaseProvider
│   │   ├── registry.py                     # Provider Factory & Dynamic Loader
│   │   ├── p_pick/                         # Penyedia Deteksi Fase P-Wave
│   │   │   ├── __init__.py
│   │   │   ├── sta_lta.py                  # Algoritma deterministik klasik (CPU)
│   │   │   ├── phasenet.py                 # Model Deep 1D U-Net (GPU/CPU)
│   │   │   └── eqtransformer.py            # Model Transformer komprehensif
│   │   ├── association/                    # Penyedia Asosiasi Gempa
│   │   │   ├── __init__.py
│   │   │   ├── dbscan_simple.py            # Algoritma spasio-temporal clustering
│   │   │   ├── gamma.py                    # Gaussian Mixture Model Association
│   │   │   └── pyocto.py                   # Algoritma asosiasi seismik densitas tinggi
│   │   └── locmag/                         # Penyedia Lokasi & Magnitudo
│   │       ├── __init__.py
│   │       ├── geiger.py                   # Penentuan hiposenter metode Geiger
│   │       └── nonlinloc.py                # Non-linear global search locator
│   │
│   ├── seedlink_module/                    # [EKSISTING] Penyerapan Data Gelombang Real-Time
│   │   ├── seedlink_consumer.py
│   │   └── Dockerfile
│   ├── archiving_module/                   # [EKSISTING] Persistensi Arsip miniSEED ke Disk
│   ├── fdsn_module/                        # [EKSISTING] Layanan Standar FDSN Web Service
│   ├── websocket_module/                   # [DITINGKATKAN] WebSocket Gateway Real-Time
│   │   ├── websocket_general.py
│   │   └── websocket_waveform.py
│   │
│   └── controller_module/                  # [LEGACY / DEPRECATED] Modul lama (dijaga sementara)
│       └── ...                             # Akan dialihkan sepenuhnya ke modul 'api/'
│
├── tews-ui-vue/                            # [DITINGKATKAN] Frontend SPA (Vue 3, TypeScript, Vite)
│   ├── src/
│   │   ├── components/
│   │   │   ├── PipelineWizard/             # [BARU] Komponen Wizard Perakitan Pipa Seismik
│   │   │   │   ├── StageSelector.vue       # Pemilihan tahapan alur data
│   │   │   │   ├── ModelSelector.vue       # Grid kartu pemilihan model AI
│   │   │   │   ├── ModelCard.vue           # Kartu rincian akurasi & tarif per model
│   │   │   │   └── ReviewPanel.vue         # Konfirmasi topologi pipa pemrosesan
│   │   │   ├── PriceCalculator/            # [BARU] Komponen Kalkulator Tarif Real-Time
│   │   │   │   └── PriceCalculator.vue     # Sidebar interaktif estimasi biaya (IDR)
│   │   │   ├── BillingDashboard/           # [BARU] Komponen Dasbor Faktur & Pemakaian
│   │   │   │   ├── UsageChart.vue          # Visualisasi tren beban komputasi harian
│   │   │   │   └── InvoiceTable.vue        # Tabel riwayat tagihan dan tombol unduh PDF
│   │   │   └── ...                         # Komponen visualisasi peta Leaflet & chart D3 eksisting
│   │   │
│   │   ├── pages/
│   │   │   ├── PipelineBuilder.vue         # [BARU] Halaman utama wizard konfigurasi profil
│   │   │   ├── Profiles.vue                # [BARU] Halaman manajemen profil aktif dan arsip
│   │   │   ├── Billing.vue                 # [BARU] Halaman pelaporan tagihan dan kuota
│   │   │   ├── Dashboard.vue               # [EKSISTING] Halaman monitoring gempa real-time
│   │   │   └── Stations.vue                # [EKSISTING] Halaman kesehatan jaringan seismik
│   │   │
│   │   ├── stores/                         # [BARU] Manajemen State Terpusat (Pinia)
│   │   │   ├── profileStore.ts             # State draf konfigurasi, validasi, dan revisi aktif
│   │   │   ├── pricingStore.ts             # State estimasi harga live dan akumulasi tagihan
│   │   │   └── catalogStore.ts             # State cache daftar model dan tarif dari server
│   │   │
│   │   └── services/                       # [BARU] Klien Komunikasi REST API (Axios Client)
│   │       ├── profileApi.ts               # Pemanggil endpoint /api/v2/profiles
│   │       ├── pricingApi.ts               # Pemanggil endpoint /api/v2/pricing
│   │       ├── catalogApi.ts               # Pemanggil endpoint /api/v2/models
│   │       └── billingApi.ts               # Pemanggil endpoint /api/v2/billing
│   │
│   └── package.json
│
├── database/                               # [BARU] Skrip Migrasi & Inisialisasi Basis Data
│   ├── migrations/
│   │   ├── 001_init_collections.js         # Skrip inisialisasi koleksi profiles, usage, invoices
│   │   └── 002_create_indexes.js           # Pembuatan indeks compound pada usage_daily
│   └── seed/
```

```
│       └── pricing_seed.json                # Data awal tarif resmi standar BMKG
│
├── docker/                                  # [BARU] Berkas Definisi Kontainerisasi Layanan Baru
│   ├── Dockerfile.api                       # Kontainer layanan API Gateway FastAPI
│   ├── Dockerfile.metering                  # Kontainer layanan penyerapan metering
│   └── Dockerfile.billing                   # Kontainer layanan generator penagihan bulanan
│
├── docker-compose.yml                       # [DITINGKATKAN] Orkestrasi seluruh multi-service produksi
├── docker-compose.wrapper.yml               # [EKSISTING] Wrapper DinD (Docker-in-Docker) BMKG Server
├── docker-compose.dev.yml                   # [BARU] Lingkungan lokal terisolasi untuk pengujian cepat
│
└── development/                             # [EKSISTING & DOKUMENTASI]
    ├── ARCHITECTURE.md                      # Desain arsitektur domain sistem bisnis
    ├── ARCHITECTURE_PAY_PER_USE.md          # Rancang bangun struktur modul penagihan
    ├── implementation_guide.md              # Panduan teknis rincian kode
    ├── pipeline_profile.py                  # Prototipe skrip domain model pipa
    ├── pricing_model.py                     # Prototipe kalkulator penetapan harga
    ├── test_pipeline_profile.py             # Unit test verifikasi logika domain
    └── LAPORAN_RENCANA_PENGEMBANGAN_ARSITEKTUR_AI_GEMPA.md # [DOKUMEN INI]
```

7. TAHAPAN IMPLEMENTASI & RENCANA KERJA (IMPLEMENTATION ROADMAP)

Rencana pengembangan dirancang dengan pendekatan bertahap (*phased rollout*) selama **5 hingga 6 minggu kerja**, di mana setiap fase menghasilkan luaran (*deliverable*) yang terverifikasi dan tidak merusak fungsi pemantauan gempa yang sedang aktif berjalan.

7.1 Matriks Fase, Durasi, dan Deliverables



Fase Pelaksanaan	Estimasi Durasi	Ruang Lingkup & Sasaran Pekerjaan	Rincian Deliverables Utama	Kriteria Keberhasilan (Definition of Done)
Fase 1: Standardisasi Katalog & Fondasi Profile Service	Minggu 1	• Pembentukan `catalog_module/` (`manifest.yaml`, `rates.yaml`). • Pembangunan `core/profile.py` & `services/profile_service.py`. • Implementasi endpoint `/api/v2/models` dan CRUD `/api/v2/profiles`.	• Berkas manifest & rates siap. • Unit test kompilasi hash profil (100% lulus). • Endpoint OpenAPI v2 aktif di Swagger UI.	Pengembang dapat membuat draf profil dan menerima kalkulasi validasi skema melalui perintah REST API.

Fase Pelaksanaan	Estimasi Durasi	Ruang Lingkup & Sasaran Pekerjaan	Rincian Deliverables Utama	Kriteria Keberhasilan (Definition of Done)
Fase 2: Pembaruan Antarmuka Pengguna (Frontend Wizard)	Minggu 2	• Pembangunan `PipelineWizard.vue` 4-tahap di frontend Vue 3. • Integrasi state Pinia (`profileStore`, `catalogStore`, `pricingStore`).• Pembuatan komponen `PriceCalculator.vue` dengan mekanisme *debounce* 500ms.	• Halaman `/pipeline-builder` aktif. • Panel sidebar menghitung estimasi biaya live saat berganti model. • Antarmuka responsif pada desktop analis.	Pengguna dapat merakit profil pipa pemrosesan secara visual dan melihat simulasi biaya IDR secara instan.
Fase 3: Infrastruktur Metering & Event Streaming	Minggu 3 s/d 4 (2 minggu)	• Pemasangan pemancar `metering_events` pada `p_pick_module` dan `association_module`. • Pembangunan kontainer `metering_module` (Kafka Consumer). • Implementasi algoritma windowing 10 menit dan persistensi ke koleksi `usage_daily`.	• Topik Kafka `metering_events` terhubung. • Kontainer `metering` beroperasi stabil di docker-compose. • Data agregasi harian tersimpan konsisten di MongoDB.	Beban inferensi model AI secara otomatis tercatat ke dalam tabel database tanpa adanya penurunan throughput seismik.
Fase 4: Sistem Penagihan & Faktur Otomatis (Billing Engine)	Minggu 5	• Implementasi `billing_module/` dan *cron job* penagihan bulanan. • Algoritma kalkulasi pemotongan kuota gratis dan penambahan PPN 11%. • Pembuatan halaman frontend `Billing.vue` dan ekspor faktur format PDF.	• Skrip cron penagihan bulanan teruji. • Koleksi `invoices` terisi data faktur tervalidasi. • Halaman pelaporan tagihan aktif di antarmuka web.	Sistem berhasil menerbitkan faktur tagihan berformat resmi yang merangkum rincian unit pemakaian dan pajak riil.

Fase Pelaksanaan	Estimasi Durasi	Ruang Lingkup & Sasaran Pekerjaan	Rincian Deliverables Utama	Kriteria Keberhasilan (Definition of Done)
Fase 5: Pengujian Akhir, Gateway Pembayaran & Rollout	Minggu 6	• Integrasi opsional webhook gerbang pembayaran (Xendit/Stripe). • Pengujian beban (*load testing*) dan simulasi *failover* jaringan. • Migrasi profil operasional BMKG eksisting ke format profil baru. • Dokumentasi panduan operasional teknis.	• Seluruh lingkungan lulus uji regresi. • Migrasi profil lama berhasil tanpa *downtime*. • Penyerahan panduan operasional kepada staf BMKG.	Seluruh alur (pembentukan profil, inferensi model, pencatatan meteran, hingga penerbitan tagihan) beroperasi mulus secara *end-to-end*.

7.2 Strategi Migrasi Tanpa Gangguan Layanan (Zero Downtime Migration)

Mengingat operasional pemantauan gempa bumi BMKG berlangsung selama 24 jam sehari, 7 hari seminggu, strategi migrasi dirancang dengan prinsip **keterpautan longgar (*loose coupling*)**:

1. **Dual Routing (API Gateway Coexistence):**
    - API controller lama (`/api/*`) tetap dipertahankan berjalan berdampingan selama masa transisi.
    - Seluruh fungsionalitas baru diarahkan ke *namespace* `/api/v2/*`.
    - Modul antarmuka lama dan aplikasi eksternal yang belum diperbarui tetap dapat berkomunikasi tanpa gangguan.
2. **Default Profile Seed (Otomasi Migrasi Awal):**
    - Saat sistem baru pertama kali dinyalakan, skrip inisialisasi akan secara otomatis mengonversi konfigurasi stasiun dan model aktif eksisting menjadi sebuah profil terdaftar: `default@rev-1`.
    - Modul inferensi akan menggunakan `default@rev-1` sebagai acuan operasional dasar sampai operator secara sadar mengaktifkan profil baru melalui wizard.
3. **Mekanisme Fallback Aman (*Failover Safeguard*):**
    - Apabila profil kustom yang baru diaktifkan mengalami anomali atau kegagalan koneksi sensor di tengah jalan, sistem pengontrol secara otomatis

memicu pengembalian (*rollback*) ke profil `default@rev-1` dan mengirimkan sinyal peringatan darurat ke dashboard analis.

8. ANALISIS RISIKO TEKNIS & RENCANA MITIGASI

Skenario Risiko	Tingkat Dampak	Probabilitas	Rencana Mitigasi Teknis Proaktif
Latensi Tambahan pada Jalur Seismik Akibat Metering	Tinggi	Rendah	Komunikasi metering dilakukan murni secara asinkron (*non-blocking fire-and-forget*) melalui Apache Kafka. Kegagalan broker metering tidak akan pernah menghentikan pipeline deteksi gempa utama.
Beban Tulis (*Write Bottleneck*) ke MongoDB Akibat Volume Data Tinggi	Sedang	Sedang	Diterapkan mekanisme penyangga (*in-memory buffering*) dengan interval agregasi 10 menit pada `metering_module`. Database hanya menerima satu transaksi tulis terakumulasi per interval, bukan satu transaksi per inferensi gelombang.
Kesalahan Parameter Seismik dari Operator di UI	Tinggi	Rendah	Form wizard dilengkapi dengan validasi skema bertingkat (*Pydantic Schema Validation*) yang menolak nilai ambang batas (*threshold*) yang tidak rasional serta mewajibkan tahap konfirmasi peninjauan sebelum profil dapat diaktifkan.
Ketergantungan terhadap Gateway Pembayaran Eksternal	Rendah	Sedang	Modul pembayaran pihak ketiga (Xendit/Stripe) dirancang bersifat modular dan *decoupled*. Layanan penagihan tetap dapat menerbitkan faktur internal BMKG secara penuh meskipun integrasi pembayaran daring dinonaktifkan via environment variable.
Fragmentasi Memori pada Lingkungan DinD (Docker-in-Docker)	Sedang	Rendah	Menggunakan volume mount terpisah untuk data Docker (`gempa-docker-data`) dan direktori arsip gelombang (`/mnt/archive`), serta menerapkan limitasi alokasi RAM per kontainer pada berkas `docker-compose.yml`.